ECE 220 Computer Systems & Programming

Lecture 15 — Data Structures

October 15, 2026

Prof. Sayan Mitra mitras@illinois.edu

Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

A Class Roster in Three Arrays

```c
/* A class roster in three parallel arrays */
#include <stdio.h>
#define N 3

int main() {
    char  Name[N][100] = {"Jamie", "Alex", "Robin"};
    int   UIN[N] = {123456789, 987654321, 246813579};
    float GPA[N] = {3.89, 3.98, 3.52};
    int   i, t;

    /* exchange students 0 and 1 */
    t = UIN[0];
    UIN[0] = UIN[1];
    UIN[1] = t;

    for (i = 0; i < N; i++)
        printf("%s %d %.2f\n",
               Name[i], UIN[i], GPA[i]);
    return 0;
}
```

A Class Roster in Three Arrays

```c
/* A class roster in three parallel arrays */
#include <stdio.h>
#define N 3

int main() {
    char  Name[N][100] = {"Jamie", "Alex", "Robin"};
    int   UIN[N] = {123456789, 987654321, 246813579};
    float GPA[N] = {3.89, 3.98, 3.52};
    int   i, t;

    /* exchange students 0 and 1 */
    t = UIN[0];
    UIN[0] = UIN[1];
    UIN[1] = t;

    for (i = 0; i < N; i++)
        printf("%s %d %.2f\n",
               Name[i], UIN[i], GPA[i]);
    return 0;
}
```

Output?

_____

_____

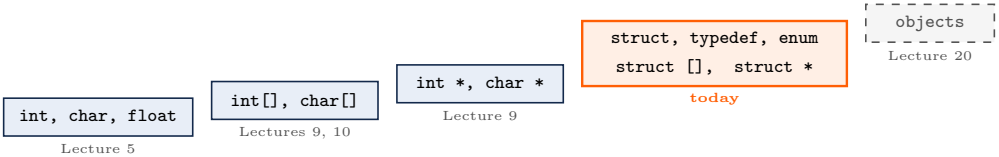
_____

Student `i` is spread over `Name[i]`, `UIN[i]` and `GPA[i]`. To exchange students 0 and 1 — the move a sort makes — the program must swap _____ times.

Nothing in this program says that the three arrays describe the same students. We need one variable that holds one student.

Today's Three Questions

A struct puts one student's data in one place.



1. How do we define a type that holds a name, a UIN and a GPA?
2. Where do the members sit in memory, and how big is one student?
3. What is copied when a student is passed to a function?

Also today: two smaller user-defined types, `enum` and `union`.

struct: One Student, One Variable

```
struct StudentStruct {
    char  Name[100];
    int   UIN;
    float GPA;
};
int main() {
    struct StudentStruct s1;
    /* declare and initialize at once */
    struct StudentStruct s2 =
        {"Alex", 987654321, 3.98};

    strncpy(s1.Name, "Jamie", sizeof(s1.Name));
    s1.UIN = 123456789;
    s1.GPA = 3.89;
```

- A **struct** is a new type built from existing types (an *aggregate* type). Its parts are its **members**.
- The dot selects a member.
  Type of `s1.UIN`: _____
  Value of `s1.Name[0]`: _____
- An initializer fills the members in order: `s2.GPA` is _____
- `Name` is an array, so `s1.Name = "Jamie";` does not compile. `strncpy(dst, src, n)` from `string.h` copies a string, at most `n` chars.

typedef: a Short Name for the Type

```
typedef struct StudentStruct {
    char  Name[100];
    int   UIN;
    float GPA;
} student;
```

- `typedef` *type name*; gives an existing type a second name
- `student s1, s2;` now declares what `struct StudentStruct s1, s2;` declared
- The same in two steps: define `struct StudentStruct {...};` and then write `typedef struct StudentStruct student;`

```
student ece220[200] = {
    {"Jamie", 123456789, 3.89},
    {"Alex",  987654321, 3.98},
    {"Robin", 246813579, 3.52}};
student t;
int i;

/* exchange students 0 and 1 */
t = ece220[0];
ece220[0] = ece220[1];
ece220[1] = t;
```

First line printed: _____

One assignment copies a whole student, `Name` included. How many bytes is that? Next: the struct in memory.

One Student in Memory

```
student s1 = {"Jamie", 123456789, 3.89};
```

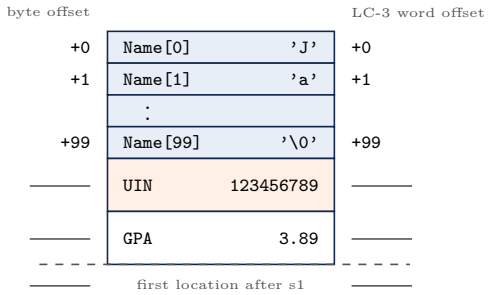
- Members sit in declaration order, one after another, at increasing addresses
- Two models of memory:

	`char`	`int`	`float`
64-bit laptop, clang	1 byte	4 bytes	4 bytes
LC-3	1 word	1 word	(1 word)

  The LC-3 has no `float`; we give `GPA` one word.

- `sizeof(student)` = _____ bytes on the laptop; on the LC-3, `s1` takes _____ words

No member name survives compilation. `s1.UIN` becomes "start of `s1`, plus 100", as a local became an offset from R5 in Lecture 5.



Lower addresses at the top.

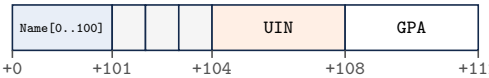
Byte offsets as printed by `sizes.c`.

sizeof and Padding

```
typedef struct {          /* one more char */
    char  Name[101];
    int   UIN;
    float GPA;
} student101;
```

```
    printf("student    %lu\n",
           sizeof(student));
    printf("student101 %lu\n",
           sizeof(student101));
```

A `student101`, byte by byte:



- `sizeof(x)` is the number of bytes `x` occupies; the compiler computes it
- `student`: $100 + 4 + 4 = 108$.
  `sizeof` prints _____
- `student101`: $101 + 4 + 4 = 109$.
  `sizeof` prints _____
- On this machine an `int` or a `float` starts at an address that is a multiple of 4. The compiler skips bytes to get there: **padding**.
- LC-3 model: every member is a whole number of words, so `student101` takes _____ words

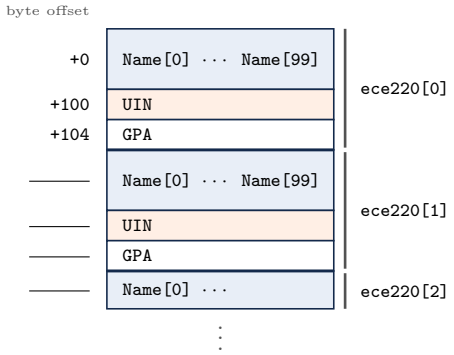
Byte counts on this slide: 64-bit laptop, clang. Write `sizeof(student)` in your code and let the compiler count.

Arrays of Structs

```
student ece220[200];

ece220[0].Name[0] = 'J';
ece220[0].Name[1] = 'a';
ece220[0].Name[2] = 'm';
ece220[0].Name[3] = 'i';
ece220[0].Name[4] = 'e';
ece220[0].Name[5] = '\0';
ece220[0].UIN = 123456789;
ece220[0].GPA = 3.89;
```

- Type of `ece220[0]`: _____
- Type of `ece220[0].Name[0]`: _____
- Why `Name[5] = '\0'`? _____
- `sizeof(ece220)` = _____ bytes



`ece220[1]` **starts one whole student after** `ece220[0]`**. A pointer to a student has to step the same distance.**

Pointer to a Struct

```
student ece220[200];
student s1;
student *s_ptr, *s_ptr2;
s_ptr = ece220;  /* to a struct array */
s_ptr2 = &s1;    /* to a struct */

strncpy(s_ptr->Name, "Jamie",
        sizeof(s1.Name));
s_ptr->UIN = 123456789;
s_ptr->GPA = 3.89;

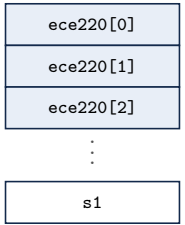
s_ptr++;
```

`s_ptr->UIN` means `(*s_ptr).UIN`: follow the pointer, then select the member.

Pointer arithmetic counts in elements (Lecture 9). Here one element is a whole student: 102 words on the LC-3.

- The three assignments changed _____
- After `s_ptr++`, `s_ptr` points at _____
- Its value grew by _____

Draw `s_ptr` before and after ++, and `s_ptr2`:



Passing a Struct: by Value, by Reference

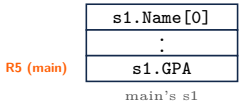
```
void printname(student s) {
    printf("%s, %lu bytes passed\n",
           s.Name, sizeof(s));
}
void PRINTNAME(student *s) {
    printf("%s, %lu bytes passed\n",
           s->Name, sizeof(s));
}
int main() {
    student s1 = {"Jamie", 123456789, 3.89};
    printname(s1);
    PRINTNAME(&s1);
    return 0;
}
```

Output:

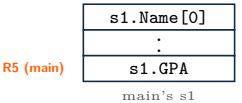
`Jamie, _____ bytes passed`

`Jamie, _____ bytes passed`

Draw each call's run-time stack (LC-3 model, Lecture 8):



`printname(s1)`



`PRINTNAME(&s1)`

Which call lets the function change `main`'s `s1`?

_____

Arguments are copied, as with `swap` (Lecture 9): a whole struct, or one address.

In-Class Problem: Highest GPA

`top_gpa(ece220, n)` returns the *address* of the record with the highest GPA among the first `n`.

- `main` reads `roster.txt` with `fscanf` (Lecture 14), then calls `top_gpa`
- What arrives in `s_ptr`? _____
- `main` prints the winner's name as _____
- For the five records in `roster.txt` it prints _____

```
student *top_gpa(student *s_ptr, int n) {
    /* best: address of the highest GPA so far */


    /* for records 1..n-1: advance s_ptr, */
    /* compare GPAs, update best          */



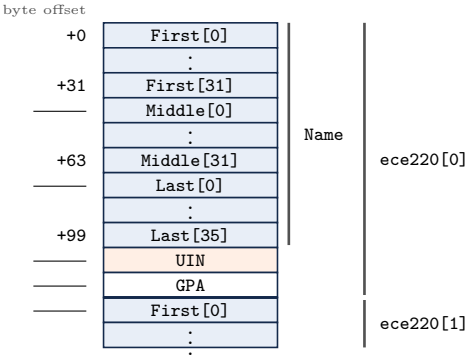

    /* hand back the address of the winner */


}
```

In-Class Problem: Struct within a Struct

```
typedef struct StudentName {
    char First[32];
    char Middle[32];
    char Last[36];
} name;
typedef struct StudentStruct {
    name  Name;
    int   UIN;
    float GPA;
} student;
int main() {
    student ece220[200];
    student *ptr;
    ptr = ece220;
```

```
    /* set First in the first record, through ptr */
```



- Type of `ptr->Name`: _____
- Type of `ptr->Name.First`: _____
- `sizeof(student)` is still _____

enum: Names for Integers

```
enum Days {SUN, MON, TUE, WED, THU, FRI, SAT};
int main() {
    enum Days today;
    today = THU;
    printf("today = %d\n", today);
    if (today == SAT || today == SUN)
        printf("weekend\n");
    else
        printf("%d days to SAT\n", SAT - today);
    return 0;
}
```

	SUN	MON	TUE	WED	THU	FRI	SAT
default							
SUN=1							

- An **enumerated type** is a user-defined list of related names, its *enumeration constants*. The count starts at 0 by default.
- `today` is stored as an `int`. Its value is _____
- The program then prints _____
- With `enum Days {SUN=1, MON, ...}` the value of `today` is _____

union: Members Share the Same Memory

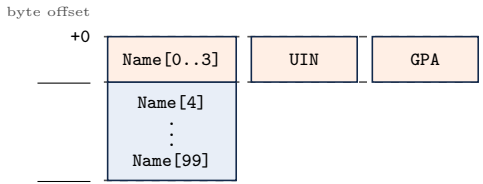
```
typedef union StudentUnion {
    char  Name[100];
    int   UIN;
    float GPA;
} studentU;
    studentU su1;
    su1.UIN = 123456789;
    printf("UIN = %d\n", su1.UIN);
    su1.GPA = 3.89;
    printf("UIN = %d\n", su1.UIN);
    printf("GPA = %f\n", su1.GPA);
```

Output: UIN = 123456789

UIN = _____

GPA = _____

- A **union** is declared like a struct, and every member starts at offset 0
- `sizeof(studentU)` = _____, the size of its largest member. The struct took 108.



A union holds one member at a time, and the program has to remember which one it wrote last.

Summary & What's Next

Today

- **The problem**: a roster in three parallel arrays keeps one student in three places
- **struct** groups members of different types into one type; `.` selects a member; `typedef` names the type `student`
- **In memory**: members in declaration order; `sizeof(student)` is 108 bytes on a 64-bit laptop, 102 words on the LC-3; padding keeps an `int` at a multiple of 4; `ece220[1]` starts one student after `ece220[0]`
- **Pointers**: `s_ptr->UIN` is `(*s_ptr).UIN`; `s_ptr++` moves one student; passing a struct copies every member, passing its address copies one word; `ptr->Name.First` reaches into a struct within a struct
- **enum** names a list of integers; a **union** overlays its members at offset 0

Next lecture

- Dynamic memory allocation: `student ece220[200]` fixes the class size at compile time. `malloc` and `free` choose it while the program runs.

Code from today: `roster_arrays.c`, `record.c`, `roster.c`, `sizes.c`, `sarray.c`, `sptr.c`, `passing.c`, `top_gpa.c` (reads `roster.txt`), `nested.c`, `days.c`, `union.c`.